SUPPLEMENT

n8n 入門與 AI Agent 實作

n8n — From Zero to Your First AI Agent

從沒用過 n8n，到自己動手做出一個會查資料、會用工具的 AI Agent。
本講義每一步都可以照著做——概念、 Windows 安裝、核心節點，到兩個完整的動手範例。

資料科學　2027 Spring　·　補充教材　·　自動化與 Agent 工具　|　謝楠楨

本講義大綱

01 n8n 是什麼、不是什麼 · 自動化工具的定位地圖

02 能自動化的六種情境 · 從行銷到醫療的實例

03 核心概念速記 · Node ／ Workflow ／ Trigger ／ Credential

04 ★ Windows 安裝全流程 · WSL2 → Docker Desktop → n8n

05 編輯器介面導覽 · Canvas 、節點面板、 NDV

06 核心節點全景 · Trigger 類與 Action 類節點

07 資料在節點間怎麼流 · Item 概念與 Expression

08 ★ 範例一：每日天氣提醒 · 逐欄位設定的完整實作

09 AI Agent 是什麼 · Agent vs Chain 的關鍵差異

10 Agent 三件套 · Chat Model • Memory • Tools

11 Agent 的思考迴圈 · Thought → Action → Observation

12 Credential 設定 · 申請並安全存放 API 金鑰

13 ★ 範例二：多工具 AI Agent Chatbot · 七個 Step 從零組出

14 除錯與 Executions · 看懂錯誤、修好它

15 ★ n8n × Dify 長照 3.0 整合 · 兩個平台串起來的完整實作

16 總結與延伸資源 · 檢核清單與下一步

學完這個單元，你要能做到

1. **在 Windows 上用 Docker 跑起 n8n，並說出 n8n 的定位與六個核心概念**
   對應： PART 01–02 · n8n 是／不是（第 5 頁）、六個核心概念（第 8 頁）、啟動（第 14 頁）

2. **看懂編輯器、節點分類與資料流，會用 Expression 取用上一個節點的資料**
   對應： PART 03–04 · 編輯器介面（第 21 頁）、資料的樣子（第 28 頁）、 Expression （第 29 頁）

3. **完成範例一「每日天氣提醒」： Trigger → 抓資料 → 判斷 → 輸出**
   對應： PART 05 · HTTP Request （第 34 頁）、 If 判斷（第 35 頁）、隨堂檢核（第 39 頁）

4. **說明 Chain 與 Agent 的差別，設好 Credential，並用 $fromAI() 做出多工具 Agent**
   對應： PART 06–08 · Chain vs Agent （第 41 頁）、 Credential （第 49 頁）、 $fromAI() （第 61 頁）

5. **用 Executions 除錯、正確發布上線，並把 Dify 長照 3.0 接進 n8n 做批次評估**
   對應： PART 09–11 · Executions （第 68 頁）、測試到上線（第 70 頁）、整合架構（第 77 頁）

需要先會： M8 的 Agent 與工具調用概念、四段式 Prompt （ PART 10 對照 n8n 與 Dify 的分工）。

PART 01

n8n 是什麼

What n8n Is

它是一個「用畫的」自動化平台——把不同服務串接起來，
中間可以放邏輯判斷，也可以放一顆會思考的 AI Agent。

n8n 是／不是

n8n 是

- ▸ 視覺化工作流程編輯器：拖節點、連線，就是一個自動化程式
- ▸ 開源、可自架（fair-code 授權），資料完全留在自己的機器上
- ▸ 支援 400 + 內建節點，幾乎所有主流服務都能直接串
- ▸ 原生支援 AI／LangChain 元件： Agent 、 Chat Model 、向量資料庫
- ▸ 程式碼救援：內建 Code 節點，邏輯太複雜時可以直接寫 JavaScript 或 Python
- ▸ workflow 可以呼叫另一個 workflow ，像函式一樣被重複使用

n8n 不是

- ▸ 不是無程式碼的萬靈丹——複雜邏輯還是需要工程思維去拆解
- ▸ 不是資料庫或儲存服務——它負責「串接與搬動」，不負責長期儲存
- ▸ 不會讓爛 API 變好——外部服務掛了， workflow 一樣會失敗
- ▸ 不是專門的 LLM 應用平台—— RAG ／知識庫這類功能不如 Dify 精緻
- ▸ 自架版不是零維運——伺服器、更新、備份還是要自己顧
- ▸ 免費版沒有多人協作、 SSO 等企業功能，那些在付費方案

定位對照： Dify （M8）專注在「組裝 LLM 應用」； n8n 的強項是「串接任何服務＋放一個 Agent 進去」，通用性更高，但 LLM 應用的專屬功能較陽春。

自動化工具定位圖



讀圖方式：越往右＝越難學；越往上＝能做的事越多。 n8n 位在右上——比 Zapier 難一些，但換來的是「幾乎什麼都串得起來」＋「可以自架」＋「能放 AI Agent」。

n8n 能幫你自動化的六種情境

案例 01

行銷與內容

- 每天自動彙整競品動態並發 Email 摘要
- 新文章發佈時自動同步到多個社群平台
- 表單有人填寫就自動加進名單並發歡迎信

案例 02

客服與支援

- AI Agent 先攔截常見問題，答不出來才轉真人
- 客訴內容自動分類並指派給對應窗口
- 滿意度回覆自動彙整成週報

案例 03

資料與報表

- 每天定時從多個系統抓資料彙整成一張報表
- 偵測到異常數值自動發出警示通知
- 定期把試算表資料同步進資料庫

案例 04

個人生產力

- 新信件自動摘要重點並分類到資料夾
- 會議紀錄自動整理成待辦清單
- 跨平台的通知統一彙整成一則每日摘要

案例 05

醫療與長照（呼應 M8／M9）

- 長照個案資料定期彙整並產出評估摘要
- 新法規公告自動比對現有流程並提醒承辦
- AI Agent 協助檢核表單完整性，缺漏立即通知

案例 06

軟體開發與維運（呼應 M10）

- 部署完成自動通知團隊並附上變更摘要
- 例行備份與健康檢查，異常自動告警
- GitHub 有新 Issue 自動分類並指派

共同的骨架都一樣： Trigger （什麼時候）→ 抓資料或接收事件 → 判斷／處理（可能交給 AI Agent ）→ 輸出或通知。學會這個骨架，就能套用到任何情境。

六個一定要先懂的核心概念

CONCEPT 01

Node 節點

- 工作流程裡的一個方塊
- 一個節點＝一個動作或判斷
- 雙擊節點可設定詳細參數

CONCEPT 02

Workflow 工作流程

- 一群節點連接起來的整體
- 從 Trigger 開始，依序執行
- 可以另存、複製、匯出成 JSON

CONCEPT 03

Trigger 觸發節點

- 工作流程「什麼時候」開始
- 每個 workflow 至少要有一個
- 形狀是藥丸狀，和一般節點不同

CONCEPT 04

Credential 憑證

- 存放 API 金鑰等認證資訊
- 建立一次，多個節點可共用
- 不會出現在匯出的 JSON 裡

CONCEPT 05

Execution 執行紀錄

- 每跑一次 workflow 就留一筆
- 可回放、可看每個節點的輸出
- 除錯的第一站

CONCEPT 06

Expression 表達式

- 用兩層大括號動態帶入資料
- 讓 workflow 不是寫死的固定值
- 是「資料會流動」的關鍵語法

這六個詞會貫穿整份講義。先有這張速記卡，後面看到專有名詞不會卡住。

PART 02

Windows 安裝全流程

Installation on Windows

本章以 Windows 10 ／ 11 為例，從「什麼都沒裝」開始，
一步一步裝到瀏覽器能打開 n8n 為止。每一步都有驗證方式。

先決定： Cloud 還是自架?

考量	n8n Cloud	自架（本章教這個）
安裝	不需要，註冊即用	需要先裝 Docker Desktop
技術門檻	幾乎不需要	需要跟著步驟操作，不需寫程式
費用	有免費試用，之後按方案付費	Community 版永久免費
資料位置	存在 n8n 的雲端	★ 完全留在你自己的電腦
適合	只想快速試玩、不想裝東西	課堂練習、專題開發、處理敏感資料

只是想先看看 n8n 長什麼樣子 → 用 Cloud 免費試用最快。打算做專題、或會處理機構／健康資料 → 直接照本章自架，資料不出自己的電腦。

Windows 安裝：先確認這四件事

項目	需要達到的條件	怎麼查
① **作業系統**	Windows 11 任何版本， 或 Windows 10 64 位元 22H2	按 Win ＋ R → 輸入 winver → Enter， 跳出的視窗會顯示版本號
② **硬體虛擬化**	64 位元 CPU 且已開啟 Intel VT-x 或 AMD-V	工作管理員 → 效能 → CPU， 看「虛擬化」是否為「已啟用」
③ **記憶體與空間**	記憶體 8GB 以上、 可用磁碟空間 6GB 以上	工作管理員 → 效能 → 記憶體； 檔案總管看 C 槽剩餘空間
④ **帳號權限**	需要系統管理員帳號	開始功能表右鍵 → 有「以系統管理員身分執行」選項即可

要裝的軟體只有兩個： WSL2 （Windows 內建功能，一行指令啟用）→ Docker Desktop。★ 不需要安裝 Node.js，也不需要會用 Linux。若「虛擬化」顯示已停用，需重開機進 BIOS／UEFI 開啟 Intel VT-x 或 AMD-V （通常在 Advanced 或 CPU Configuration 底下）。

Step 1 ｜安裝 WSL2

Windows 的 Linux 子系統， Docker 需要它才能運作

以「系統管理員」身分開啟 PowerShell： 按下 Windows 鍵 → 輸入 powershell →
在「 Windows PowerShell 」上按右鍵 → 選「以系統管理員身分執行」→ 出現藍底視窗即可。

在 PowerShell （系統管理員）貼上這一行

```
wsl --install
```

▶ 這一行會自動安裝 WSL2 與預設的 Ubuntu ，過程約 5–10 分鐘
▶ 安裝完成後，畫面會要求你「重新啟動電腦」——請務必重開機，否則 Docker 會裝不起來
▶ 重開機後會自動跳出一個黑色視窗要你設定 Linux 使用者名稱與密碼；
　隨意設定即可（例如 user ／自訂密碼），這組帳密與 n8n 無關，之後幾乎用不到
▶ 若出現「找不到 wsl 命令」，代表 Windows 版本太舊，請先執行 Windows Update

驗證：重開機後在 PowerShell 輸入　wsl -l -v　應該看到 Ubuntu 且 VERSION 欄位是 2

Step 2 | 安裝 Docker Desktop

n8n 會跑在 Docker 容器裡，環境乾淨、不會弄亂你的電腦

1	① 下載	到 docker.com 首頁點 Download Docker Desktop，選 Windows 版（AMD64 或 ARM64 依你的 CPU）
2	② 安裝	雙擊下載的 Docker Desktop Installer.exe；安裝選項中確認勾選「Use WSL 2 instead of Hyper-V」
3	③ 重開機	安裝完成後再重新啟動一次電腦
4	④ 啟動並同意條款	開啟 Docker Desktop，接受服務條款；個人與教學用途免費，可跳過登入帳號
5	⑤ 等待就緒	左下角的鯨魚圖示由「橘色轉動中」變成「綠色 Engine running」才算完成

驗證：開啟 PowerShell 輸入 `docker --version` 應該印出版本號（例如 Docker version 2x.x.x）

Step 3 ｜啟動 n8n

複製貼上兩行指令， n8n 就跑起來了

① 先建立一個資料夾（volume），用來保存你的 workflow

```
docker volume create n8n_data
```

② 啟動 n8n （PowerShell 請把行末的 ^ 換成反引號 ` ；或直接把五行接成一行）

```
docker run -d --name n8n -p 5678:5678 ^
  -e GENERIC_TIMEZONE="Asia/Taipei" ^
  -e TZ="Asia/Taipei" ^
  -v n8n_data:/home/node/.n8n ^
  docker.n8n.io/n8nio/n8n
```

▶ 第一次執行會自動下載 n8n 映像檔，約 1–3 分鐘（之後啟動只需幾秒）

▶ -p 5678:5678　把容器的 5678 埠對應到你的電腦，瀏覽器才連得到

▶ -v n8n_data:...　把資料存進剛剛建立的 volume，重開機資料不會不見

驗證：瀏覽器開啟　http://localhost:5678　會看到 n8n 的帳號設定畫面

更省事的做法：完全用 Docker Desktop 圖形介面

不想打指令也可以

- Docker Desktop 上方搜尋框輸入 n8nio/n8n → 找到官方映像檔 → 按 Pull
- 切到 Images 分頁 → 在 n8nio/n8n 這一列按 Run

Run 之前展開 Optional settings，填三個欄位

- Container name： n8n　　Host port： 5678
- Volumes： Host path 選一個你自己的資料夾， Container path 填 /home/node/.n8n
- Environment variables：新增 GENERIC_TIMEZONE = Asia/Taipei

啟動與日後管理

- 按 Run 後，到 Containers 分頁就能看到 n8n 正在執行，按停止／啟動都在這裡
- ★ 這個做法和上一頁的指令完全等價，只是換成用滑鼠點

建議：第一次先用圖形介面把它跑起來建立信心；等到需要重複佈署或寫進文件時，再改用指令比較快。

Windows 安裝常見問題排除

症狀	原因	處方
wsl --install 顯示找不到命令	Windows 版本太舊	執行 Windows Update 更新到 22H2 以上，再重試
Docker 啟動後一直轉圈／報 WSL 錯誤	WSL2 沒裝好或沒重開機	PowerShell 執行 wsl --update，然後重新啟動電腦
安裝時提示虛擬化未啟用	BIOS 沒開 VT-x ／ AMD-V	重開機進 BIOS，找 Intel VT-x 或 SVM Mode 並啟用
localhost:5678 打不開	容器沒啟動，或埠號被佔用	Docker Desktop 看容器是否 Running； 或把 -p 改成 5679:5678 換一個埠
重開機後 workflow 不見了	啟動時沒有掛 volume	確認指令有 -v n8n_data:/home/node/.n8n
排程時間差 8 小時	時區沒設定	確認有帶 GENERIC_TIMEZONE=Asia/Taipei

卡關時的通則：先看 Docker Desktop 的 Containers 分頁確認容器狀態，再點該容器的 Logs 看錯誤訊息——九成問題在這裡就能定位。

Step 4 ｜第一次進入 n8n

開啟檔案

瀏覽器開啟：

`http://localhost:5678`

（若換過埠號，
請改成你設定的埠）

預期結果

- ▸ 能成功登入並看到 Overview 工作流程列表
- ▸ 點 Create Workflow 後進入空白畫布
- ▸ ★ 到這裡安裝全部完成，接下來就是純粹的操作練習了

操作步驟

1. 第一次進入會要求建立擁有者帳號：填 Email 、姓名、密碼
2. ★ 這組帳密只存在你自己的電腦，與 n8n 官方無關，忘記時要用 n8n user-management:reset 指令重設
3. 接著出現問卷詢問用途，可以直接跳過（ Skip ）
4. 可選：畫面會提示免費註冊解鎖 Folders 等額外功能，填信箱即可，仍然免費
5. 進入 Overview 頁面後，點右上角 Create Workflow 建立第一個工作流程
6. 看到中間一片空白的畫布與「 Add first step 」按鈕，代表環境完全就緒

日常操作：怎麼開、怎麼關、怎麼更新

每天要用的時候

- 開啟 Docker Desktop → Containers 分頁 → 在 n8n 這列按「▶ 啟動」
- 或在 PowerShell 輸入： docker start n8n

用完想關掉

- 同一個位置按「停止」，或輸入： docker stop n8n
- ★ 停止不會刪掉資料， workflow 都還在

要更新到新版

- docker pull docker.n8n.io/n8nio/n8n （下載新版）
- docker stop n8n → docker rm n8n → 重新執行 Step 3 的啟動指令
- 因為資料存在 volume 裡，更新後 workflow 與憑證都會保留

n8n 大約每週釋出新版。課堂期間不必追每週新版，避免步驟對不上；但看到安全公告要當天更新（見下一頁）。

實例：2026 年初的 Ni8mare 漏洞

發生了什麼（2026/1/7 公開）

- CVE-2026-21858 （暱稱 Ni8mare ）， CVSS 10.0 滿分：攻擊者不用登入，透過對外公開的 Form ／ Webhook 就能接管自架的 n8n
- 影響 1.121.0 以前的自架版本；資安研究估計約 10 萬台伺服器暴露在網路上
- 前後幾週還有多個 9.9–10 分的漏洞（如 CVE-2026-21877 、 CVE-2025-68613 ）——一次修一個不夠，要跟上版本

為什麼跟你有關

- n8n 的 Credentials 存著你所有的 API 金鑰；被接管等於所有串接的服務一起外洩
- 範例二的 Chat Trigger 公開連結、 Webhook ，都是「對外開放的入口」
- 2026/10 的 3.0 版更進一步：自架只支援 Docker 、移除 Function 等舊節點——舊教學的步驟會失效

三個習慣

- 每月照上一頁的步驟 docker pull 更新一次；看到安全公告當天就更新
- 課堂練習只開在 localhost ，不要把自架的 n8n 直接開到網際網路；要給別人用時加帳號保護並限制來源
- 處理健康資料的流程，先問資訊室：機構有沒有統一管理與更新的環境

資料： Cyera Research Labs 、 Rapid7 、 Censys 2026/1 漏洞公告； n8n 官方 v3.0 breaking changes 文件（ 3.0 測試版 2026/10/27 ，正式版預定 12 月初）。

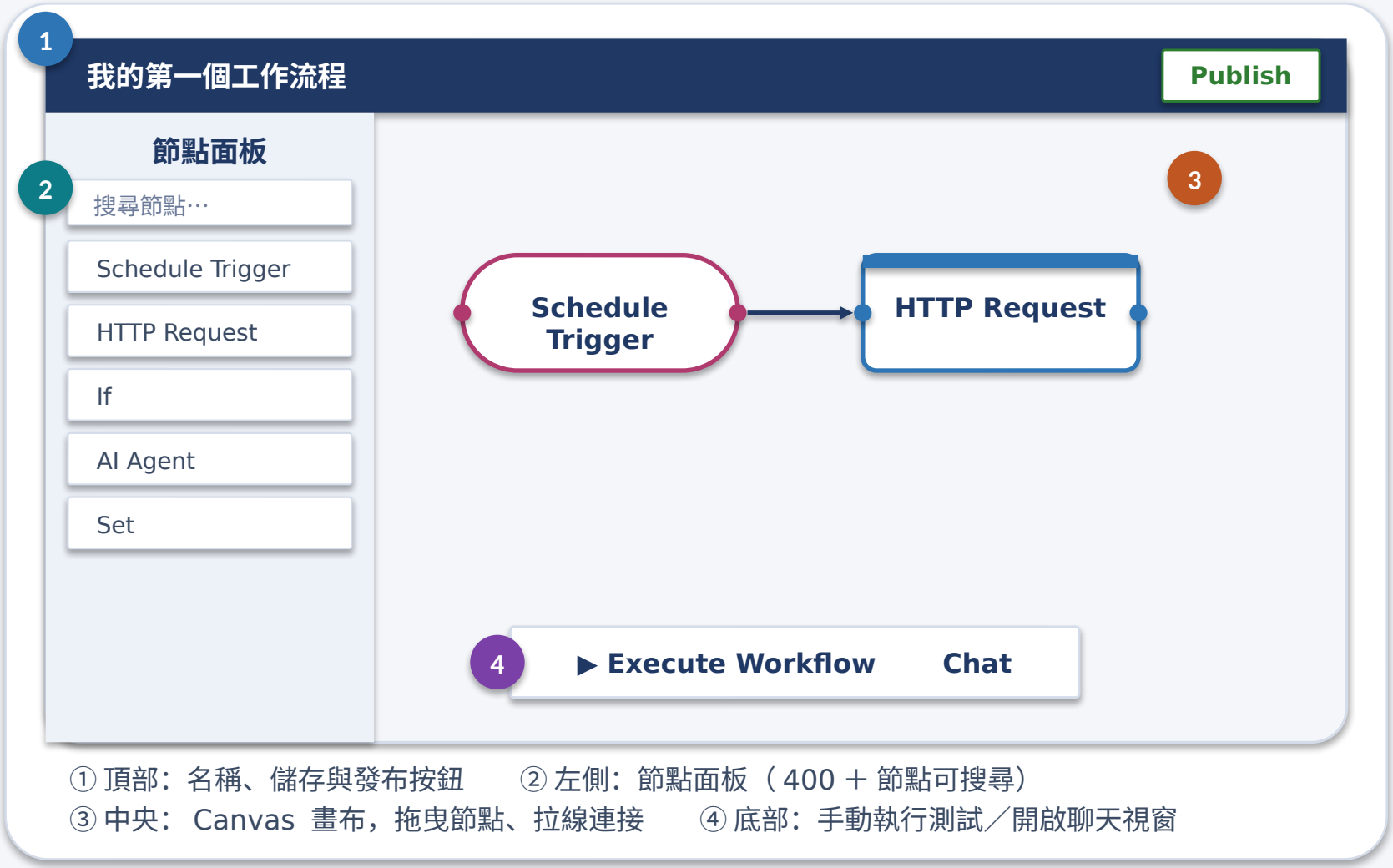
PART 03

編輯器介面與核心節點

The Canvas & Core Nodes

裝好了、打開了，接下來要看懂畫面上每一塊在做什麼，
以及有哪些節點是「幾乎每個 workflow 都會用到」的。

n8n 編輯器介面導覽



① 頂部：名稱、儲存與發布按鈕　② 左側：節點面板（400 + 節點可搜尋）
③ 中央：Canvas 畫布，拖曳節點、拉線連接　④ 底部：手動執行測試／開啟聊天視窗

① 頂部

工作流程名稱、儲存狀態、右上角 Publish 發布按鈕（2.0 前叫 Active）

② 左側

節點面板——可搜尋 400 + 內建節點，中英文都能搜

③ 中央

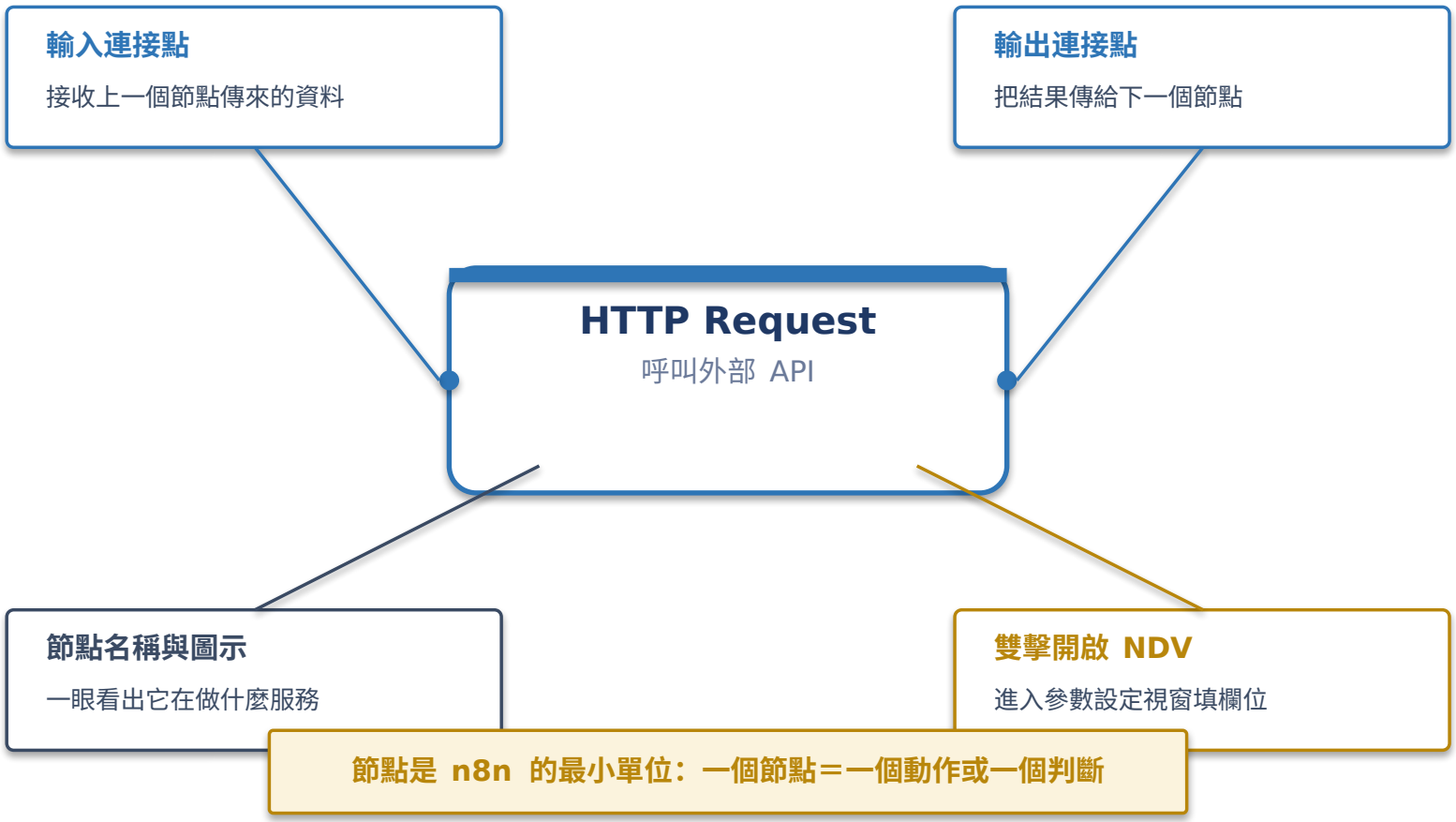
Canvas 畫布——拖曳節點、從圓點拉線連接

④ 底部

手動執行測試；AI workflow 會多一個聊天視窗

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

拆解一個節點



輸入連接點

左側圓點，接收上一個節點傳來的資料

輸出連接點

右側圓點，把處理後的結果傳給下一個節點

節點名稱

可以自己改名，建議取有意義的名字方便日後閱讀

雙擊開啟 NDV

Node Detail View：左邊看輸入、中間填參數、右邊看輸出

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

Trigger 節點 vs 一般節點



★ 每個工作流程一定要有至少一個 Trigger 節點。
它決定「什麼時候」開始執行；後面接的所有節點都在回答「該做什麼」。

怎麼分辨?

左右兩端都圓（藥丸狀）＝ Trigger

一個流程可以有幾個?

至少 1 個；多個 Trigger 各自獨立啟動

Trigger （藥丸狀）

決定 workflow 「什麼時候」開始執行

Action （方角）

決定「該做什麼」——處理、判斷、輸出

規則

每個可執行的 workflow 至少要有一個 Trigger

小提醒

一個 workflow 可以有多個 Trigger，各自獨立啟動

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

節點的三大分類

1 Core Nodes 核心邏輯節點

- 不綁定特定服務，處理邏輯與資料
- Schedule ／ Webhook ／ Manual Trigger
- If 、 Switch ： 條件分支
- Set ： 新增／修改欄位
- Code ： 需要時寫 JavaScript 或 Python
- Merge 、 Filter ： 合併與篩選資料

2 App Nodes 服務整合節點

- 每個節點對應一個外部服務
- Gmail 、 Slack 、 Telegram 、 Notion...
- Google Sheets 、 Airtable 、 資料庫類
- 共 400 + 個，且持續增加
- 多數需要先設定 Credential 才能用
- 找不到對應節點時， HTTP Request 幾乎萬用

3 AI Nodes LangChain 節點

- 本講義的重點： AI Agent 相關節點
- AI Agent ： root node ，決策中樞
- Chat Model 系列：接不同 LLM 供應商
- Memory 系列：記住對話
- Tool 系列：讓 Agent 「會做事」
- Vector Store ： 進階的 RAG 知識庫

找節點最快的方法：在節點面板直接打關鍵字。想不到節點名稱時，先想「我要的是哪一個服務」，多數服務名稱直接搜就有。

常用 Trigger 節點速查

節點	觸發時機	常見用途
Manual Trigger	按下 Execute Workflow 手動觸發	開發階段測試用
Schedule Trigger	依排程（每天／每週／ Cron ）	定期報表、定時通知
Webhook	外部服務打一個 URL 進來	串接第三方系統、接收表單
Chat Trigger	使用者在聊天視窗輸入訊息	★ AI Agent 對話型應用的入口
Form Trigger	使用者填寫 n8n 產生的表單	不需另外架網頁的輕量表單收集
Email Trigger (IMAP)	收到新郵件時	自動分類、摘要、建立待辦

常用 Action 節點速查

節點	做什麼	常見用途
HTTP Request	呼叫任何 API	★ 串接沒有專屬節點的服務；查天氣、查匯率
Set （Edit Fields）	新增／修改／整理欄位	把上一步的資料整理成你要的格式
If	二選一的條件分支	依條件走不同路徑
Switch	多選一的條件分支	依類別分成 3 條以上路徑
Code	寫 JavaScript ／ Python 處理資料	邏輯太複雜、節點做不到時的救援
Merge	合併多條分支的資料	把 If ／ Switch 分開的路徑合回一條
Loop Over Items	把多筆資料分批處理	避免一次打爆外部 API 的速率限制

新手策略：先想「我要做的事，能不能用現成節點兜出來」，真的兜不出來才用 Code 節點——維護成本差很多。

PART 04

資料怎麼在節點之間流動

How Data Flows

這是新手最容易卡住的地方。
搞懂「上一個節點輸出什麼、下一個節點怎麼取用」，後面就一路順了。

n8n 的資料長什麼樣子

每個節點輸出的都是一個「陣列」，裡面是一筆一筆的 item

- 就算只有一筆資料，它也是一個只有一個元素的陣列
- NDV 右側 OUTPUT 面板可以切換 Table ／ JSON 兩種檢視，建議先看 JSON 認識結構

每個 item 裡面有 json 與 binary 兩部分

- json：一般的資料欄位（文字、數字、巢狀結構），這是最常用的
- binary：檔案類內容（圖片、 PDF 等），處理附件時才會用到

★ 節點會對「每一筆 item」各執行一次

- 上一步輸出 10 筆，下一個 HTTP Request 節點就會被呼叫 10 次
- 這是 n8n 最重要的心智模型——也是為什麼要小心外部 API 的速率限制

卡關時的萬用解法：把上一個節點的 OUTPUT 切到 JSON 檢視，照著實際看到的層級一層一層寫下來，就不會猜錯欄位路徑。

Expression 表達式

上一個節點的輸出（JSON）

```
{
  "city": "Taipei",
  "temp": 28,
  "rain_prob": 62
}
```

拖曳

節點參數欄位（Expression 模式）

```
{{ $json.city }} 目前
{{ $json.temp }} 度
```

↓ 實際算出來的值

Taipei 目前 28 度

1. ▶ fx 表達式讓參數「動態」帶入前面節點的資料，而不是寫死固定值
2. ▶ 語法固定是兩層大括號，中間放 JavaScript 或 n8n 專用語法（如 $json 、 $now ）
3. ▶ 點參數欄位旁的 fx 圖示切換到表達式模式，可即時預覽算出來的值

語法

固定用兩層大括號包住，裡面放 JavaScript 或 n8n 專用語法

最常用

$json 代表「上一個節點輸出的這一筆資料」

怎麼寫

直接把左側 INPUT 的欄位拖進參數框， n8n 會自動生成表達式

即時預覽

切到 Expression 分頁能立刻看到算出來的值，不用整個執行

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

Expression 常用語法速查

語法	意思	實際例子
$json. 欄位名	上一個節點輸出的某個欄位	$json.city
$json.a.b[0]	巢狀結構與陣列取值	$json.daily.precipitation_probability_max[0]
$(' 節點名稱 ').item.json.x	指定某個節點（不一定是上一個）的輸出	$('HTTP Request').item.json.temp
$now / $today	目前時間／今天日期	$now.format('yyyy-MM-dd HH:mm')
$today.minus(7,'days')	日期運算	算出 7 天前的日期
三元運算 ? :	條件判斷	$json.temp > 30 ? ' 炎熱 ' : ' 舒適 '

不用背語法。多數情況拖曳欄位、用滑鼠點選就能完成；只有取陣列元素（加 [0] ）與條件判斷需要自己動手改一點點。

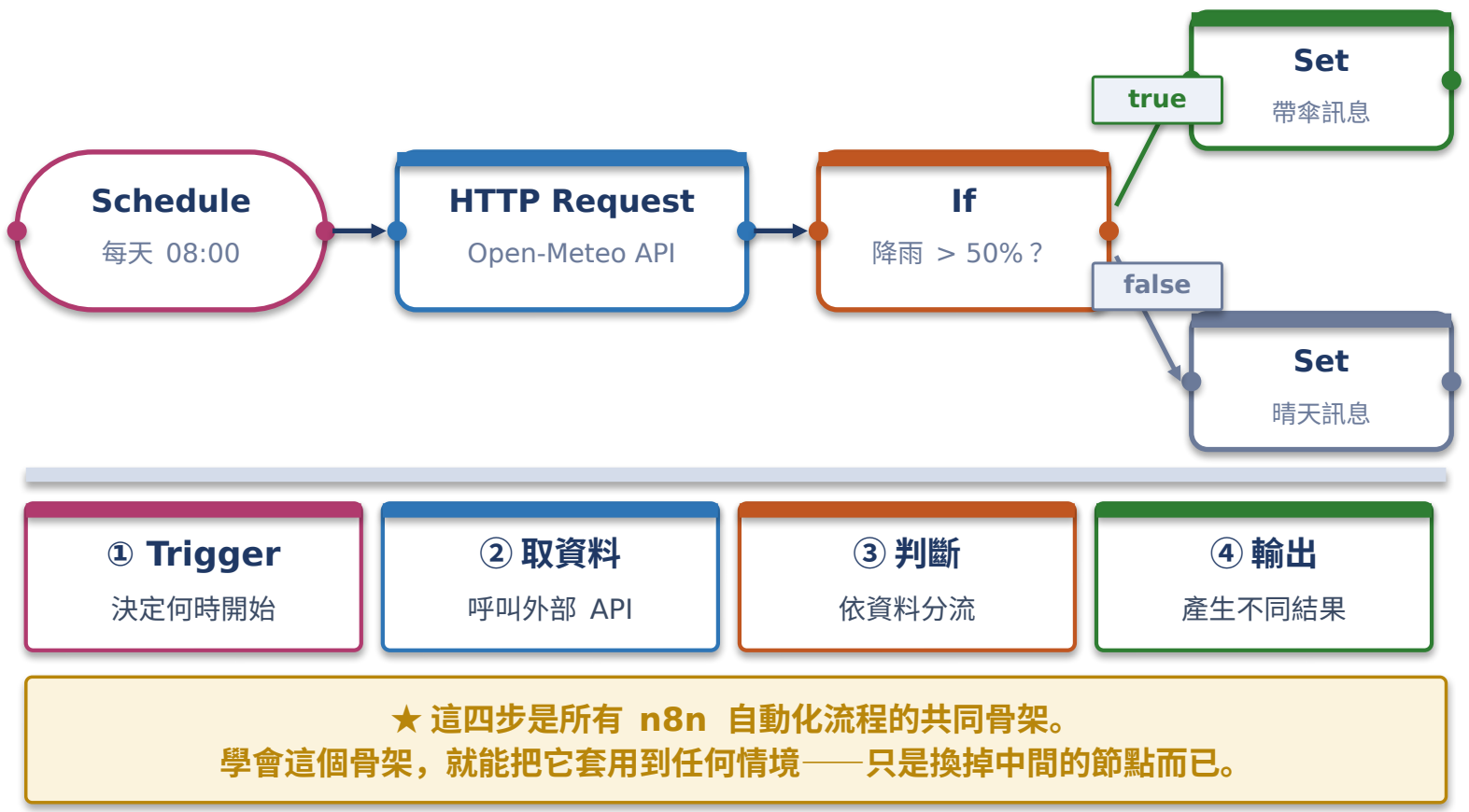
PART 05

範例一：每日天氣提醒

Workflow 1 — Daily Weather Alert

不需要任何帳號、任何金鑰，人人都能做完。
做完你會擁有「定時 → 抓資料 → 判斷 → 輸出」這個萬用骨架。

範例一：完成後長這樣



為什麼選這個

不需要任何付費帳號或 API 金鑰，人人可做

練到的技能

Trigger、HTTP Request、Expression、If 分支，一次到齊

資料來源

Open-Meteo—— 免金鑰的開放天氣 API

預計時間

照著做約 15–20 分鐘

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

範例一　Step 1 ｜建立 workflow 與排程觸發

開啟檔案

畫面位置：

```
左上角 Overview
→ Create Workflow

畫布中央
Add first step
```

預期結果

- ▸ 畫布上出現一個粉紅色藥丸狀的 Schedule Trigger 節點
- ▸ 節點下方顯示排程摘要，類似「Every day at 8:00」
- ▸ ★ 此時它還不會真的執行——要按 Publish 發布才會（本範例先不發布，用手動測試）

操作步驟

1. 進入 n8n 首頁，右上角按 Create Workflow
2. 點畫面正中央的「Add first step...」
3. 搜尋框輸入 Schedule，選擇「Schedule Trigger」
4. 節點會被加到畫布上並自動開啟設定視窗（NDV）
5. 設定 Trigger Rules： Trigger Interval 下拉選「Days」
6. Days Between Triggers 填 1（每天一次）
7. Trigger at Hour 選「8am」、Trigger at Minute 填 0
8. 左上角把 workflow 名稱改成「每日天氣提醒」，按 Ctrl + S 儲存

範例一　Step 2 ｜ HTTP Request 節點：逐欄位設定

這一頁是全講義最重要的操作頁——照著每個欄位填

HTTP Request 節點設定內容

【基本設定】

```
Method              GET
URL                 https://api.open-meteo.com
                    /v1/forecast
```

【開啟查詢參數】
把 Send Query Parameters 開關打開
然後用 Add Parameter 逐一新增：

```
  latitude     25.03        （台北緯度）
  longitude    121.56       （台北經度）
  daily        precipitation_probability_max
  current      temperature_2m
  timezone     Asia/Taipei
  forecast_days  1
```

預期輸出（數值每天不同，結構固定）

【按 Execute step 後，OUTPUT 應該長這樣】

```
{
  "latitude": 25.0,
  "longitude": 121.5,
  "timezone": "Asia/Taipei",
  "current": {
    "time": "2027-03-10T08:00",
    "temperature_2m": 24.3
  },
  "daily": {
    "time": ["2027-03-10"],
    "precipitation_probability_max": [ 62 ]
  }
}
```

★注意 daily 底下的值是「陣列」，取值要加 [0]

▶ 若要改成其他城市，只要換掉 latitude ／ longitude 兩個數字即可（可用 Google 地圖右鍵查座標）
▶ 若 OUTPUT 出現紅色錯誤，先檢查 URL 有沒有多空格、參數名稱有沒有拼錯

範例一　Step 3 ｜ If 節點：設定判斷條件

這一步決定 workflow 要走哪一條路

If 節點設定

【新增節點】
點 HTTP Request 右側的 ＋ 號
搜尋 If，選擇「If」節點

【Conditions 條件設定】

左值 (Value 1)　切到 Expression 模式，填：
```
{{ $json.daily
   .precipitation_probability_max[0] }}
```

資料型別　選 Number
運算子　　選 is greater than
右值 (Value 2)　填 50

為什麼要加 [0]？

▶ API 回傳的 precipitation_probability_max 是一個「每日一格」的陣列

▶ 我們只要今天這一格，所以取第 0 個元素（程式的計數從 0 開始）

▶ 不加 [0] 的話，比較的會是整個陣列而不是數字，條件永遠不成立

★ 這是新手最常踩的坑：看 OUTPUT 的 JSON 檢視，只要看到中括號 [ ]，取值就要加索引

設定完按 Execute step 測試：OUTPUT 會分成 true 與 false 兩個分頁，
資料會落在其中一邊。若今天不會下雨，資料就會出現在 false 分頁——這是正常的。

▶ 想立刻測試另一條分支？把右值 50 暫時改成 0（幾乎一定成立）或 100（幾乎一定不成立），測完再改回來
▶ If 節點的兩個輸出點：上面是 true、下面是 false，拉線時要注意別接反
▶ 需要三條以上分支時，改用 Switch 節點

範例一　Step 4 ｜ Set 節點：產生兩種訊息

開啟檔案

要組出的結構：

```
      If
   ├─true→ Set A
   └─false→ Set B
```

（兩個 Set 節點
設定方式相同，
只有文字不同）

預期結果

- ▸ 畫布上呈現一個分岔結構： If 之後分成上下兩條路
- ▸ 點畫面下方 Execute Workflow 跑完整流程
- ▸ 只有符合條件的那一條分支會執行並顯示綠色勾勾
- ▸ ★ 點開執行到的 Set 節點， OUTPUT 應該看到一個組好的完整中文句子

操作步驟

1. 點 If 節點「true」輸出點的 ＋ 號，搜尋 Set，選「Edit Fields (Set)」
2. 在 Fields to Set 按 Add Field
3. Name 填 message， Type 選 String
4. Value 切到 Expression，填：今天降雨機率 {{ $json.daily.precipitation_probability_max[0] }}%，記得帶傘
5. 把這個節點改名為「帶傘提醒」（雙擊節點標題即可改名）
6. 回到 If 節點，點「false」輸出點的 ＋ 號，同樣新增一個 Set 節點
7. 同樣新增 message 欄位， Value 填：今天降雨機率低，放心出門。目前氣溫 {{ $json.current.temperature_2m }} 度
8. 改名為「晴天提醒」， Ctrl ＋ S 儲存

範例一：卡關時對照這張表

症狀	可能原因	處方
HTTP Request 回傳錯誤	URL 打錯或參數名稱拼錯	逐字對照 Step 2；注意 open-meteo 中間是連字號
If 節點永遠走同一邊	左值忘了加 [0]，比較的是整個陣列	改成 ...precipitation_probability_max[0]
If 節點報型別錯誤	資料型別選成 String 卻拿來比大小	把 Condition 的型別改成 Number
Set 節點的 message 是空的	Value 沒有切到 Expression 模式	把滑鼠移到欄位上，點出現的 Expression 分頁
訊息裡顯示 [object Object]	取到的是整個物件不是單一數值	回 OUTPUT 的 JSON 檢視確認完整路徑
Execute Workflow 沒反應	Trigger 是 Schedule，需手動執行	直接按畫布下方的 Execute Workflow 按鈕

讓它真的有用：三個延伸方向

① 真的收到通知

- 把 Set 節點換成（或後面接） Gmail 、 Telegram 、 Slack 、 Line 等節點
- 把訊息內容欄位填成表達式 {{ $json.message }} 就能沿用剛才組好的句子
- Telegram ／ Line 需先申請 Bot Token 並建立 Credential

② 加上錯誤處理

- HTTP Request 節點的 Settings 分頁 → 開啟 Retry On Fail （重試次數預設 3 次）
- workflow 設定中可指定 Error Workflow ，任何節點失敗都自動通知你

③ 發布後才真的排程

- 測試沒問題後，按右上角 Publish （ 2.0 前叫 Active 開關）
- ★ Schedule Trigger 只有在 workflow 已發布時才會真的定時執行
- 自架版還需要確保 Docker 容器持續運作，電腦關機就不會執行

到這裡你已經具備組出任何「定時／觸發 → 抓資料 → 判斷 → 輸出」流程的能力。這是 n8n 最基本、也最泛用的骨架。

隨堂檢核｜介面、資料流與範例一

Q 1

Trigger 節點和一般節點最關鍵的差異是什麼?

Trigger 決定 workflow 「何時」開始；每個可執行的 workflow 至少要有一個 Trigger 。

Q 2

上一個節點輸出 10 筆 item ，下一個 HTTP Request 節點會被執行幾次?

10 次——節點會對每一筆 item 各執行一次，這是 n8n 最重要的心智模型。

Q 3

為什麼取降雨機率要寫 [0] ?

因為 API 回傳的是一個陣列（每日一格），要取今天這一格；不加索引會拿到整個陣列。

Q 4

Set 節點的 Value 填了表達式卻顯示原始文字，最可能是什麼原因?

忘記把欄位切換到 Expression 模式， n8n 把它當成純文字處理。

PART 06

AI Agent 是什麼

What an AI Agent Is

前面組的 workflow，路徑都是你自己畫死的。
Agent 不一樣——路徑由 LLM 自己想，你只負責給它腦袋跟工具。

Chain 鏈 vs Agent 代理人

比較項目	Chain （鏈）	Agent （代理人）
怎麼決定下一步	照你畫好的固定順序，一步接一步	LLM 自己判斷，動態決定要不要呼叫工具
記憶	不能用持久記憶，每次都是全新的	可以接 Memory ，記得對話上下文
可預測性	高——永遠照同一條路徑走	較低——同樣的問題可能走不同路徑
成本	可預估，每次呼叫次數固定	浮動——它可能重複呼叫工具好幾輪
除錯難度	低	較高，需要讀懂它的思考過程
適合	任務單純、路徑固定、要求穩定	任務需要「判斷」與「組合多種能力」
n8n 對應節點	Basic LLM Chain	AI Agent

不是「Agent 比較高級所以都用 Agent」。能用 Chain 說清楚的任務，用 Chain 更穩定、更好除錯、成本也更可控——這條原則和 M8 學過的 Dify 選型邏輯完全一致。

重要： n8n 版本演進（避免被舊教學搞混）

本講義以目前版本為準

1.x 時期	1.82	2.0 （2025/12）	3.0 （2026/10 測試版、12 月正式版）
AI Agent 要先選 Agent 類型（ReAct 等多種）	AI Agent 節點統一為 Tools Agent	「Active 開關」改為「Publish」修改後要再按一次才更新線上版	自架只支援 Docker（不再用 npx）移除 Function 節點與舊版 Agent

看到這些，就是舊教學——步驟會對不上

要你「選擇 Agent Type」

→ 1.82 以前的教學

畫面上有 Active 開關

→ 2.0 以前的教學

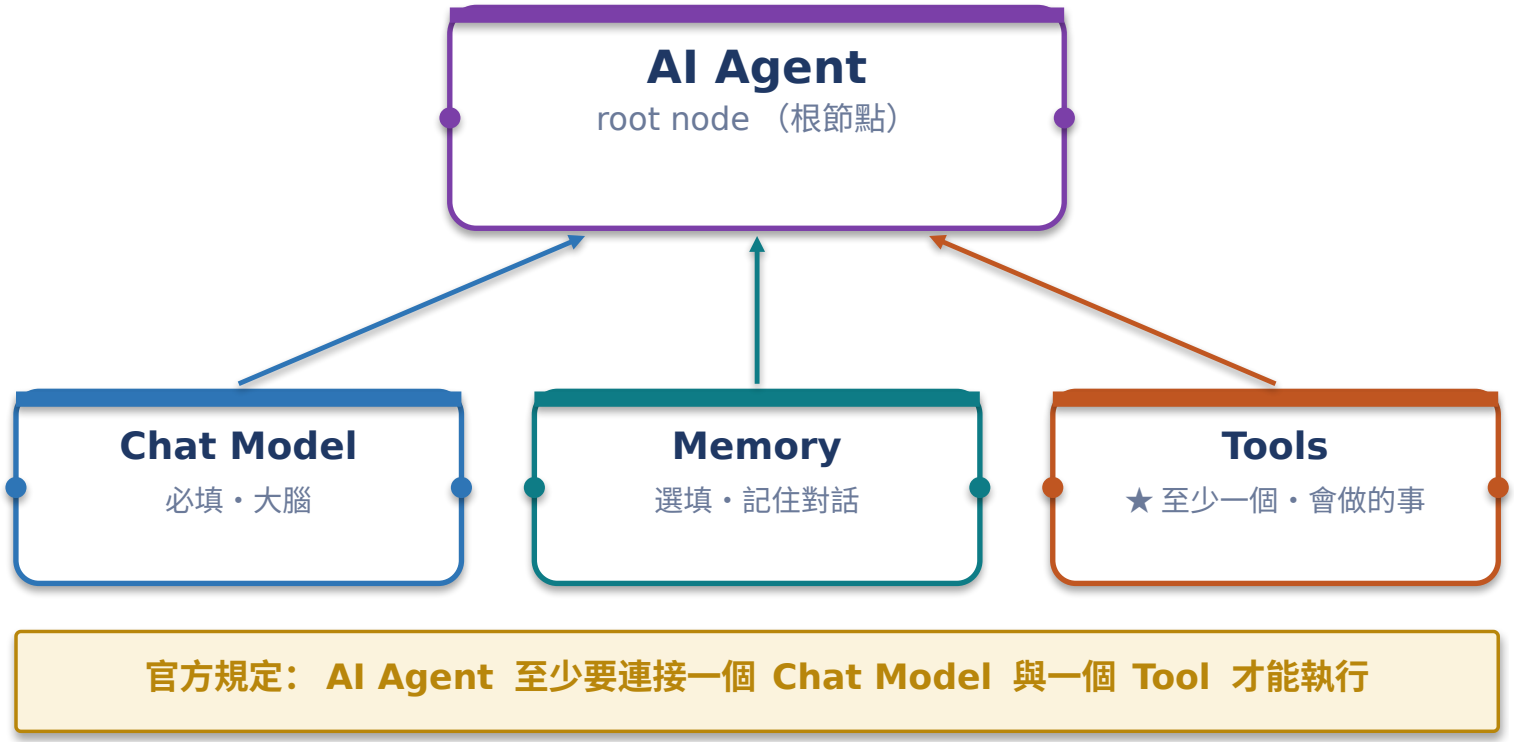
用 npx n8n 安裝，或用 Function 節點

→ 3.0 以前的教學

若看到教學要你「選擇 Agent Type」或用 npx 安裝，就是舊版教學，步驟會對不上。本講義以目前版本為準：專心把 Chat Model、Memory、Tools 接對即可。

AI Agent 節點解剖： root node ＋三種 sub node

子節點（sub node）用專屬插槽接在根節點下方，補足 Agent 的能力



Chat Model （必填）

Agent 的大腦，負責理解與生成語言，決定要不要呼叫工具

Memory （選填）

記住之前的對話，讓多輪對話有上下文

Tools （★ 至少一個）

官方規定： AI Agent 必須連接至少一個 Tool 才能執行

接法

子節點用專屬插槽接在 Agent 節點下方，不是用一般連接線

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

Chat Model 該選哪家?

供應商	費用	設定難度	適合
Google Gemini	★ 有免費額度（2025/12 起縮減，常調整）	低——Google AI Studio 直接申請	★ 課堂推薦首選，適合新手起步
OpenAI（GPT 系列）	需先儲值，按用量計費	低——申請 Key 即可用	生態最成熟，多數教學以此為準
Anthropic Claude	需先儲值，按用量計費	低	需要較強推理與長文本時
Groq	★ 免費額度寬鬆、速度極快	低	要求低延遲的即時對話場景
Ollama（本機）	★ 完全免費	中——需先在本機安裝並下載模型	資料不能出機器、或不想花錢時（M7 已裝過）

本講義的範例二示範用 Google Gemini（有免費額度、設定最快）；若你已經按 M7 裝好 Ollama，後面會附上切換成本機模型的做法。

Agent 的思考迴圈



① 使用者提問

訊息進入 Agent 節點

②③④ 核心循環

Thought（思考）→ Action（呼叫工具）→ Observation（取得結果）

⑤ 判斷

資訊夠不夠回答？不夠就回到②，換個工具或換個問法再試

⑥ Final Answer

夠了，生成最終回覆給使用者

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

Tools 的四種主要類型

類型	它是什麼	什麼時候用
內建服務 Tool	n8n 提供的現成工具： Wikipedia 、SerpAPI （搜尋）、 Calculator 等	最快上手，免設定或只需簡單憑證；先用它建立信心
★ HTTP Request Tool	把「任何」有公開 API 的服務包成 Agent 能呼叫的工具	最重要也最泛用；查天氣、查藥品、查法規都靠它——範例二會示範
Custom Code Tool	自己寫一段 JavaScript ／ Python 當工具	需要複雜運算或特殊資料處理時
Call n8n Workflow Tool	把「另一個完整 workflow 」整個包成一個工具	背後是十幾個節點的複雜流程，但 Agent 只看到「一個工具」

新手建議路徑：先用內建工具與 HTTP Request Tool 把基本功練熟，需要複雜邏輯時再進階到 Code Tool 或 Workflow Tool 。

寫好 Tool Description 的四個原則

原則	為什麼	範例（好 vs 壞）
說清楚「做什麼」	Agent 靠這段話判斷要不要用它	壞：查資料　　好：查詢指定城市的即時天氣與降雨機率
說清楚「何時用」	避免該用的時候不用、不該用時亂用	好：當使用者詢問任何城市的天氣狀況時使用
說清楚「不能做什麼」	收斂範圍，減少誤用	好：本工具只提供天氣，不能查詢空氣品質或颱風警報
用使用者會講的詞	Agent 是拿使用者原句去比對的	使用者說「會不會下雨」，描述裡就該出現「下雨／降雨」

★ Agent 不用你的工具時，九成不是模型笨，而是 Description 寫得不夠清楚。這一頁的四個原則，是排除該問題最有效的檢查清單。

PART 07

Credential 設定

Setting Up Credentials

Agent 要有大腦，大腦需要 API 金鑰才能運轉。
這一章教你申請金鑰、安全地存進 n8n 。

Credential 設定四步驟

一次設定，之後所有節點都能重複引用

① 申請金鑰
到服務商網站申請免費 API 金鑰

→

② 複製金鑰
一長串英數字，只顯示一次，先存好

→

③ 建立憑證
在 n8n 貼上金鑰並取一個好記的名字

→

④ 節點中選用
節點的 Credential 欄位下拉選擇它

金鑰只存在你自己的 n8n 裡，加密保存；匯出 workflow 的 JSON 檔不含金鑰，可安全分享

建立一次，到處可用
同一組 Credential 可被多個節點引用；金鑰換了只要改一個地方

不想花錢?
改用 Ollama 本機模型：完全免費，且資料完全不離開你的電腦

★ 千萬不要把金鑰直接打在節點參數裡——一定要走 Credential

① 申請
到服務商網站免費申請 API 金鑰

② 複製
金鑰通常只顯示一次，先存到安全的地方

③ 建立 Credential
n8n 左側選單 → Credentials → 新增，貼上金鑰並命名

④ 節點引用
Chat Model 節點的 Credential 欄位下拉選擇它

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

動手做｜申請 Google Gemini 免費 API 金鑰

Google AI Studio　申請最快、有免費額度（常調整，以官網為準）

開啟檔案

需要準備:

一個 Google 帳號
（沒有的話先申請）

申請網址:
aistudio.google.com

預期結果

- n8n 顯示連線成功（綠色勾）或未報錯即可
- 之後在任何 Chat Model 節點的 Credential 欄位，都能下拉選到「我的 Gemini」
- ★ 金鑰不會出現在匯出的 workflow JSON 中，可放心把 workflow 分享給同學

操作步驟

1. 瀏覽器開啟 https://aistudio.google.com/ ，用 Google 帳號登入
2. 左側選單找到「Get API key」（或中文介面的「取得 API 金鑰」）
3. 點「Create API key」，選一個 Google Cloud 專案（沒有的話用它預設建立的即可）
4. 複製產生的金鑰字串（一長串英數字），先貼到記事本存好
5. 回到 n8n：左側選單 Credentials → Add Credential
6. 搜尋並選擇「Google Gemini(PaLM) Api」
7. 把金鑰貼到 API Key 欄位，上方名稱改成「我的 Gemini」，按 Save

金鑰安全與免費替代方案

金鑰管理原則（與 M8 Dify 完全一致）

- 每人申請自己的金鑰，不共用、不寫進報告、不上傳公開儲存庫
- n8n 的 Credential 加密存放在本機，匯出 workflow 不含金鑰
- ★ 絕對不要把金鑰直接打在節點的參數欄位裡——一定要走 Credential

完全免費的替代方案： Ollama 本機模型

- M7 已教過安裝方式，本機已有 Ollama 可直接沿用
- Docker 版 n8n 要連本機 Ollama ， Base URL 需填 http://host.docker.internal:11434
- 優點：完全免費、資料不出機器；缺點：能力通常不如雲端大模型、回應較慢

★ 特別注意：如果 n8n 跑在 Docker 裡，容器內的 localhost 指的是「容器自己」，不是你的電腦。所以要連本機 Ollama 必須改用 host.docker.internal—— 這是新手最常見的連線失敗原因。

隨堂檢核｜AI Agent 概念與 Credential

Q 1

Agent 和 Chain 最根本的差異是什麼?

Chain 照固定順序執行； Agent 由 LLM 自己判斷下一步要做什麼、要不要呼叫工具。

Q 2

AI Agent 節點必須連接哪些子節點才能執行?

至少一個 Chat Model ，以及官方規定的至少一個 Tool ； Memory 為選填。

Q 3

Agent 一直不使用你新接的工具，最該檢查什麼?

Tool 的 Description—— 它是 Agent 判斷「何時該用這個工具」的唯一依據。

Q 4

Docker 版 n8n 要連本機 Ollama ， Base URL 該填什麼?

http://host.docker.internal:11434—— 容器內的 localhost 指的是容器自己，不是你的電腦。

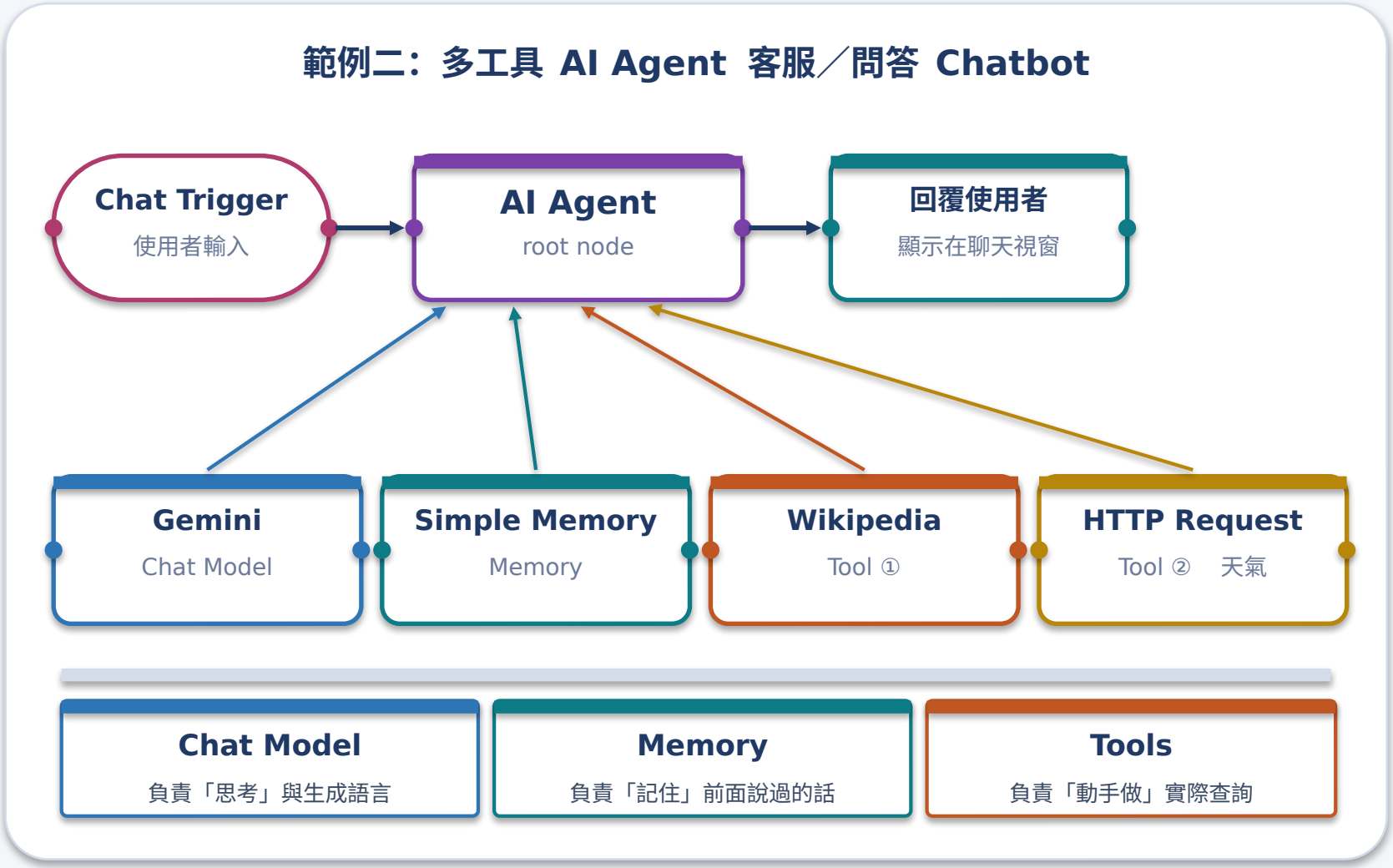
PART 08

範例二：多工具 AI Agent Chatbot

Workflow 2 — Your First Multi-Tool Agent

前面學的元素，這裡全部組在一起。
做完這個範例，你會有一個會查百科、會查天氣、記得住對話的聊天機器人。

完成品長這樣：完整節點圖



Chat Trigger

使用者在聊天視窗輸入的入口

AI Agent

中樞——接收訊息、決定要不要用工具、生成回覆

四個子節點

Chat Model 大腦、 Memory 記憶、兩個 Tools 會做的事

完成後可以問

「牛頓是誰」「東京天氣如何」，Agent 會自己選對工具

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

開始前：七個 Step 的地圖

Step	要做的事	完成後的樣子
Step 1	建立骨架： Chat Trigger ＋ AI Agent	畫布上兩個節點， Agent 顯示紅色警告
Step 2	接上 Chat Model （Gemini）	紅色警告消失一個， Agent 有了「大腦」
Step 3	接上 Memory （Simple Memory）	Agent 能記住上一句話
Step 4	接上第一個 Tool （Wikipedia）	紅色警告全部消失，可以執行了
Step 5	接上第二個 Tool （HTTP Request 查天氣）	★ 學會把任意 API 變成 Agent 的工具
Step 6	撰寫 System Message	規範角色、使用時機與誠實原則
Step 7	測試對話並讀懂 Agent 的思考	確認它會自己挑對工具

預計時間：照著做約 30–40 分鐘。前四步是把 Agent 「組起來」，第五步之後才是真正有趣的地方。

Step 1 ｜建立骨架： Chat Trigger ＋ AI Agent

開啟檔案

要組出的結構：

```
Chat Trigger
      |
  AI Agent
（三個插槽
 都還空著）
```

預期結果

- 畫布上有兩個節點並已連線
- AI Agent 節點下方出現三個子節點插槽
- Chat Model 與 Tool 顯示紅色驚嘆號——接下來三步就是把它們補滿

操作步驟

1. 新建 workflow，左上角命名為「我的第一個 AI Agent」
2. 點「Add first step...」→ 搜尋 Chat → 選「When chat message received」（Chat Trigger）
3. 此節點不需任何設定，維持預設即可，關閉 NDV
4. 點 Chat Trigger 右側 ＋ 號 → 搜尋 AI Agent → 選「AI Agent」
5. 雙擊 AI Agent 節點，會看到節點下方多出三個插槽
6. Chat Model 與 Tool 兩個插槽帶紅色驚嘆號（表示必填未設定）
7. Memory 插槽沒有驚嘆號（選填），先不管它
8. Ctrl ＋ S 儲存

Step 2 ｜接上大腦： Google Gemini Chat Model

Agent 的「思考能力」來自這裡

Chat Model 子節點設定

【操作路徑】
AI Agent 節點下方
→ Chat Model 插槽的 ＋ 號
→搜尋 Gemini
→選「Google Gemini Chat Model」

【欄位設定】

Credential　選「我的 Gemini」
（上一章建立的）
Model　gemini-3.5-flash
（或清單中最新的 flash 版）

其餘選項維持預設

為什麼選 flash 版?

▶ flash 系列是「輕量快速」版：回應快、免費額度消耗慢

▶ Agent 一次對話可能呼叫模型好幾輪，用重量級模型成本會快速累積

▶ 一般問答、工具呼叫的判斷， flash 等級完全夠用

★ 實務策略（同 M8）：判斷類節點用小模型，需要長篇生成時才切大模型

驗證：關閉子節點後回到 AI Agent ， Chat Model 插槽的紅色驚嘆號應該消失，並顯示模型名稱

▶ 想改用完全免費的本機模型？把這個子節點刪掉，改接「Ollama Chat Model」
Base URL 填 http://host.docker.internal:11434 （Docker 版 n8n）

▶ Agent 主體完全不用改——這就是子節點設計的好處：換大腦不用重做流程

Step 3 ｜接上記憶： Simple Memory

開啟檔案

要組出的結構：

```
AI Agent
   |
Memory 插槽
   |
Simple Memory
```

預期結果

- Memory 插槽出現 Simple Memory 節點
- 連續發問時， Agent 會記得前面說過的內容
- ★ 限制：只存在記憶體， n8n 重啟即清空；要永久保存需改用 Postgres ／ Redis Chat Memory

操作步驟

1. 點 AI Agent 節點下方 Memory 插槽的 ＋ 號
2. 搜尋 Memory ，選擇「 Simple Memory 」（也可能顯示為 Window Buffer Memory ）
3. Session ID 維持預設「 Connected Chat Trigger Node 」——它會自動用聊天視窗的 session
4. Context Window Length 維持預設 5 （記住最近 5 輪對話）
5. 若希望記得更久可以調大，但要注意每輪對話都會把歷史一起送給模型， Token 會增加
6. 關閉節點，確認 Memory 插槽已顯示 Simple Memory

Step 4 ｜接上第一個工具： Wikipedia

開啟檔案

要組出的結構：

```
AI Agent
   |
 Tool 插槽
   |
Wikipedia
```

預期結果

- ▸ 紅色驚嘆號全部消失， workflow 可以正常執行
- ▸ 問知識性問題時， Agent 會呼叫 Wikipedia 並根據查到的內容回答
- ▸ ★ 到這裡你已經有一個能運作的 AI Agent 了；下一步是加入第二種能力

操作步驟

1. 點 AI Agent 節點下方 Tool 插槽的 ＋ 號
2. 搜尋 Wikipedia ，選擇「 Wikipedia 」工具
3. ★ 這個工具不需要任何設定，也不需要 Credential ，維持預設即可
4. 關閉節點，回到 AI Agent ， Tool 插槽的紅色驚嘆號應已消失
5. 此時整個 workflow 已經可以執行了——點下方 Chat 按鈕試著問：「牛頓是誰？」
6. 觀察 Agent 是否呼叫了 Wikipedia 工具（回覆中會顯示工具執行標記）

Step 5 ｜第二個工具：把天氣 API 變成 Agent 的工具

★ 本範例的教學重點——學會這一步，你就能把任何 API 變成工具

HTTP Request Tool 完整設定

【操作路徑】
AI Agent → Tool 插槽的 ＋ 號 → 搜尋 HTTP Request
→ 選「HTTP Request Tool」（注意不是一般的 HTTP Request）

【工具識別資訊】節點名稱改成 get_weather
Description（給 Agent 看的說明，決定它何時使用）：
查詢指定城市目前的天氣與今日降雨機率。
當使用者詢問任何城市的天氣、氣溫或會不會下雨時，
使用這個工具。本工具不提供空氣品質或颱風警報。

【請求設定】
Method GET
URL https://api.open-meteo.com/v1/forecast

【Query Parameters（開啟 Send Query Parameters）】

```
latitude    {{ $fromAI("lat", "城市的緯度", "number") }}
longitude   {{ $fromAI("lon", "城市的經度", "number") }}
current     temperature_2m
daily       precipitation_probability_max
timezone    auto
forecast_days  1
```

這裡發生了什麼事?

▶ 使用者問「東京天氣如何?」

▶ Agent 讀 Description，判斷這題該用 get_weather

▶ Agent 看到參數是 $fromAI()，於是自己想出東京的
經緯度（約 35.68 / 139.69）並填進去

▶ n8n 帶著這組參數去呼叫 Open-Meteo

▶ API 回傳 JSON， Agent 讀懂後用自然語言回覆

★ 關鍵：城市不是你寫死的，是 Agent 自己決定的

$fromAI()：讓 Agent 自己決定要傳什麼參數

面向	說明
語法	$fromAI(key, description, type, defaultValue)
key （必填）	參數代稱；只能用英數字、底線、連字號——不能有空格或中文
description	給 Agent 的提示文字，說明這個參數代表什麼（強烈建議填）
type	string ／ number ／ boolean ／ json ，預設為 string
怎麼運作	這些不是「取值」而是「提示」： Agent 依上下文自己想出該填的值； 找不到線索時甚至會主動反問使用者「請問你要查哪個城市？」
★ 使用限制	只在「連接到 AI Agent 的 Tool 節點」裡有效；一般節點寫了不會有作用
操作捷徑	多數工具參數欄位旁有星星圖示，點下去會自動生成表達式，不用手打

這是整份講義最值得記住的一招：有了 $fromAI() ，任何有公開 API 的服務——查藥品、查法規、查床位、查給付標準——都能在十分鐘內變成 Agent 的工具。

Step 6 ｜撰寫 System Message

同 M8 的四段式 Prompt：角色・任務時機・限制・格式

System Message 範例（可直接複製使用）

位置：AI Agent 節點 → Options → Add Option → System Message

```
你是一個誠實、有幫助的中文助理，能查詢維基百科與即時天氣。

【使用工具的時機】
- 使用者詢問人物、地點、歷史、科學等知識性問題時，使用 Wikipedia 工具。
- 使用者詢問天氣、氣溫、會不會下雨時，使用 get_weather 工具。
- 一般閒聊或你已經確定知道的常識，不需要呼叫工具。

【回答原則】
- 一律使用繁體中文回答。
- 引用工具查到的內容時，簡短說明資料來源（例如：根據維基百科／根據即時天氣資料）。
- 不確定、或工具查不到時，明確說「我查不到這項資訊」，不要編造。
- 不提供醫療診斷、用藥建議或法律意見。
```

★ 這一段和 M8 教的『System Prompt 四段式』是同一套道理：角色、任務與時機、限制、格式。
差別只在 Dify 寫在提示詞編輯器、n8n 寫在 System Message 欄位——概念完全共通。

Step 7 ｜測試對話並讀懂 Agent 的思考

開啟檔案

測試位置：

畫布下方
「Chat」按鈕

開啟後在右側
輸入訊息

預期結果

- ▸ 五個測試都得到合理回覆，工具選擇正確
- ▸ 追問「那台北呢」仍知道在問天氣（Memory 生效）
- ▸ 展開執行細節可看到傳給工具的經緯度確實是該城市

操作步驟

1. 輸入：「牛頓是誰？」→ 應該呼叫 Wikipedia 工具
2. 輸入：「東京現在天氣如何？」→ 應該改呼叫 get_weather 工具
3. 追問：「那台北呢？」→ 測試 Memory： Agent 應該知道你還在問天氣
4. 輸入：「幫我推薦一部電影」→ 測試它會不會亂用工具（正確行為是不呼叫工具直接回答）
5. 輸入：「請問火星上今天的降雨機率？」→ 測試誠實原則，應該說查不到
6. ★ 點對話訊息旁的執行標記，可展開看到 Agent 實際呼叫了哪個工具、傳了什麼參數、拿回什麼結果

範例二：卡關時對照這張表

症狀	可能原因	處方
AI Agent 顯示紅色驚嘆號無法執行	Chat Model 或 Tool 插槽沒接	官方規定兩者皆為必填，各接上至少一個
Agent 一直不用新接的工具	Tool 的 Description 寫得不夠清楚	補上「何時使用」與使用者會講的關鍵詞
$fromAI() 沒作用或報錯	寫在非 Tool 節點，或 key 用了中文／空格	確認節點是接在 Agent 的 Tool 插槽；key 只用英數字與底線
Memory 感覺沒有生效	不在同一個聊天 session	同一個 Chat 視窗連續對話；換分頁會視為新對話
回覆是英文	System Message 沒有指定語言	明確寫「一律使用繁體中文回答」
Agent 一直重複呼叫工具停不下來	缺少迭代上限	Options → Max Iterations 設 5–10，避免無限打轉與費用暴增

讓別人也能用：發布你的 Chatbot

① 產生公開連結

- 雙擊 Chat Trigger 節點 → 打開「Make Chat Publicly Available」開關
- 會產生一個公開聊天網址，可直接分享；也提供 <iframe> 程式碼可嵌入自己的網頁

② 按下 Publish 正式發布

- ★ 沒按 Publish，公開連結不會生效——這是最常見的「明明做好卻沒反應」原因
- 自架版還需確保 Docker 容器持續運作；電腦關機連結就會失效

③ 公開前務必檢查

- 公開＝任何拿到網址的人都能用你的金鑰對話，會消耗你的免費額度
- 先設好 Max Iterations，並確認 System Message 的限制寫得夠嚴謹再公開

嵌入網站的做法和 M10 教過的部署是互補技能： M10 教你把整個網站部署上線，這裡教你把一個 AI 客服嵌進任何一個網頁角落。

隨堂檢核｜範例二　多工具 AI Agent

Q1　AI Agent 節點的哪兩個插槽是官方規定的必填?

Chat Model 與 Tool（至少一個）； Memory 為選填。

Q2　$fromAI() 的 description 參數有什麼作用?

它是給 Agent 的提示，讓 Agent 自己判斷這個參數該填什麼值。

Q3　為什麼 Docker 版 n8n 連本機 Ollama 要用 host.docker.internal ?

容器內的 localhost 指的是容器自己，不是你的電腦。

Q4　Agent 一直重複呼叫工具停不下來，該設哪個參數?

Options 中的 Max Iterations ，設定思考迴圈的最大輪數。

Q5　要讓對話紀錄在 n8n 重啟後仍保留，該換掉哪個子節點?

Simple Memory—— 它只存在記憶體中；改用 Postgres 或 Redis Chat Memory 。

PART 09

除錯、 Executions 與正式上線

Debugging & Going Live

Workflow 不會永遠順利。

知道去哪裡看錯誤、怎麼判斷問題出在哪，比會用所有節點更重要。

Executions 執行紀錄：除錯第一站

Executions　執行紀錄

◯ 成功	今天 08:00:03	0.8s	
× 失敗	昨天 07:00:02	0.3s	點開查看 ▶
◯ 成功	前天 08:00:01	0.7s	

失敗詳情： HTTP Request 節點

錯誤訊息： Timeout — 逾時未回應（API 端沒有回覆）
可能原因：網址打錯、外部服務暫時掛掉、網路不通
處方：檢查 URL → 開啟 Retry On Fail → 必要時延長逾時秒數

★ 每個節點的輸入與輸出都能在這裡逐一攤開檢視——這是除錯永遠的第一站

列表

每次執行都留一筆紀錄，成功／失敗一目了然

點開失敗的那筆

可以看到是哪個節點出錯、錯誤訊息是什麼

逐節點檢查

每個節點的輸入與輸出都能個別攤開檢視

常見錯誤

Timeout （逾時）、Authentication （憑證錯誤）、資料格式不符

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

除錯三步驟

① 看 Executions 列表

- 先確認是「哪一次」執行失敗，還是每次都失敗
- 偶爾失敗通常是外部服務不穩定；每次都失敗通常是設定錯誤

② 點開失敗節點，讀錯誤訊息

- n8n 的錯誤訊息通常會指出是哪個參數或哪一步出問題
- Authentication 或 401 開頭的錯誤，九成是 Credential 沒設對或已過期

③ 對照輸入與輸出

- 點開該節點前一步的 OUTPUT，確認資料格式是不是它預期的樣子
- ★ 很多錯誤其實是「上一步給的資料長相，跟這一步想要的不一樣」

AI Agent 節點的除錯多一招：點開對話中每一步的執行標記，能看到 Thought ／ Action ／ Observation 的完整過程，比看一般節點更直覺。

從測試到正式上線

測試模式　Test URL

- workflow 尚未發布
- 每次要按「Listen for Test Event」才會接收一次請求
- 適合開發階段反覆除錯

Publish

正式模式　Production URL

- 按下右上角 Publish 後生效
- URL 固定、24 小時持續接收請求
- 自架版需保持伺服器持續運作
- 這才是給真實使用者用的網址

★ 開發時用 Test URL 反覆除錯；確定沒問題後才按 Publish，拿 Production URL 給真正的使用者或要串接的服務使用。

Schedule Trigger

未發布就不會定時執行

Webhook / Chat

未發布則 Production URL 無效

忘記按 Publish

★ 最常見的「明明做好卻沒動」原因

怎麼讀這張圖

左邊是測試模式（Test URL），右邊是正式模式（Production URL）；按下 Publish 才會從左變到右。

最容易誤解的一點

Test URL 要手動按監聽才收一次；Production URL 要先 Publish。沒發布，排程與 Webhook 都不會真的跑。

對專題的意義

開發時用 Test URL 除錯，確定沒問題再 Publish；給真實使用者的網址一律用 Production URL。

圖：課程自製（本圖為 PowerPoint 原生圖形，可直接編輯）

上線前檢查清單

項目	檢查內容
Credential	所有節點用到的憑證都已設定，且不是共用帳號
錯誤處理	關鍵節點是否設定 Retry On Fail； workflow 是否掛了 Error Workflow
Publish 發布	確認已發布；之後每次修改都要再按 Publish，存檔不會更新線上版本
成本評估	AI Agent 每次對話會呼叫 LLM 多次——留意用量與免費額度
速率限制	外部 API 多半有呼叫頻率上限，批次處理要用 Loop Over Items 節流
敏感資料	涉及機構或個人資料時，確認符合自架與資安規範（同 M8 ／ M9 規範）
容器持續運作	自架版：電腦關機或容器停止，排程與 Webhook 都不會運作

PART 10

n8n vs Dify：怎麼選、怎麼一起用

Choosing & Combining Tools

兩個工具你都學過了。
這一章先講怎麼選，再用長照 3.0 實際示範怎麼把兩個串起來。

n8n vs Dify　全面對照

比較項目	n8n	Dify
設計初衷	通用自動化平台，串接任何服務	LLM 應用組裝平台，專注 Chatbot ／ Agent
學習曲線	中，節點多元、彈性大	低，介面更聚焦在對話式應用
RAG ／知識庫	有，但需要自己組裝向量流程	★ 內建完整，上傳文件即可用
串接外部服務	★ 400 ＋ 節點，幾乎什麼都能接	有限，多半靠 HTTP 節點
自動化／排程	★ 原生支援 Schedule 、 Webhook 等	較弱，非其設計重點
批次處理	★ 強項：迴圈、分批、重試都是內建節點	弱，主要為單次對話設計
最適合	跨系統自動化、資料搬動、排程任務	LLM 應用、知識問答、對話型服務

經驗法則：任務主體是「跟很多不同系統打交道」選 n8n ；任務主體是「基於知識庫回答問題」選 Dify 。兩者都很強的情境，其實可以兩個一起用——下一節就實作給你看。

適用情境對照：這個需求該用哪一個？

實務需求	建議工具	理由
長照個案的智慧評估問答系統	Dify	核心是知識庫檢索＋專業判斷， RAG 是主戲
每天自動彙整住院人次並寄報表	n8n	核心是排程＋跨系統資料搬動，沒有語言理解需求
衛教文章自動同步到三個社群平台	n8n	純粹的多服務串接，一個 LLM 都不需要
化療副作用衛教問答機器人	Dify	要掛衛教手冊知識庫並標註來源段落
★ 一次替 200 位個案跑完長照初評	n8n ＋ Dify	Dify 做單筆專業判斷， n8n 負責批次、重試與寫回
新法規公告後自動比對並通知承辦	n8n ＋ Dify	n8n 監看與通知， Dify 做語意比對與摘要
內部工具的自然語言查詢介面	Dify	單一對話入口， Dify 的 Agent 設定最快
每晚把新文件自動送進知識庫	n8n	排程＋檔案處理是 n8n 強項； Dify 1.10 起才有簡易排程

★ 判斷口訣：問「這件事的主角是『知識與對話』，還是『流程與系統』？」主角是知識與對話 → Dify；主角是流程與系統 → n8n；兩者都是主角 → 兩個一起用。

PART 11

實作： n8n × Dify 長照 3.0 批次評估

Integration Case — LTC 3.0 Batch Assessment

把 M8 做好的長照 3.0 Chatflow，變成一條每天自動跑 200 筆的產線。
Dify 負責「單筆的專業判斷」， n8n 負責「批次、重試、寫回與通知」。

為什麼要串起來？先看單用一個工具的極限

只用 Dify 的話

- 長照 3.0 Chatflow 做得再好，也只能「一次一個人、手動輸入、手動複製結果」
- 沒有排程、沒有批次、無法自動把結果寫回機構的表單或資料庫

只用 n8n 的話

- n8n 也能接 LLM，但要自己組知識庫、自己管 Prompt、自己寫精算邏輯
- 等於把 M8 已經做好且驗證過的東西再做一次，而且品質不會比較好

★ 串起來之後

- Dify 端完全不用改——它已經是一個可被呼叫的 API 了
- n8n 端只需一個 HTTP Request 節點，就能把整套專業判斷變成流水線的一站

這是實務上最常見的架構：把「專業能力」封裝在 Dify，把「流程與整合」交給 n8n。兩邊各自可獨立維護、獨立測試。

整合架構：六個節點

1

Schedule Trigger

每天 02:00 自動啟動

2

讀取待評估個案清單

Google Sheets 或資料庫節點

3

Loop Over Items

一次處理 1 筆

避免打爆 API

4

★ HTTP Request

把上個節點的變數送進長照 Chatflow

5

解析並寫回結果

Set 整理 → 寫回試算表

6

異常通知承辦

失敗或金額異常才發通知

★ 注意第 3 個節點：Dify 免費方案有並發上限，一次打 200 筆一定會拿到 429。用 Loop Over Items 一筆一筆送，是這個架構能穩定運作的關鍵。

Step 1 ｜ Dify 端：把長照 3.0 Chatflow 變成 API

Dify 的每個應用發布後，本身就是一個 REST API

開啟檔案

Dify 端要做的事：

① 發布應用
② 取得 API 金鑰
③ 記下三個
表單變數 Key

（Dify 流程本身
完全不用修改）

預期結果

- ▸ 拿到一組 app- 開頭的 API 金鑰
- ▸ 知道自己的 API 伺服器位址
- ▸ ★ 金鑰等同於這個應用的通行證，不可外流、不可寫進報告或上傳 GitHub

操作步驟

1. 在 Dify 開啟「台灣長照 3.0 Chatflow」應用
2. 右上角點「發布 Publish」→「更新發布 Publish Update」，確認是已發布狀態
3. 左側選單點「存取 API （API Access）」
4. 頁面右上角點「API 金鑰 API Key」→「建立金鑰 Create secret key」
5. 複製產生的金鑰（格式類似 app-xxxxxxxxxxxx），先存到安全的地方
6. 同一頁記下 API 伺服器位址：雲端版是 https://api.dify.ai/v1；自架版是你自己的網址
7. ★ 回到 Chatflow 的「開始」節點，確認三個變數 Key：
8. transcript、has_foreign_caregiver、income_status——n8n 端要一字不差地對應

Step 2 ｜ n8n 端： HTTP Request 節點逐欄位設定

這一頁是整個整合的核心——把三個表單變數送進 Dify

HTTP Request 節點完整設定

```
【基本設定】
Method    POST
URL       https://api.dify.ai/v1/chat-messages

【Authentication】Generic → Header Auth
  Name    Authorization
  Value   Bearer app-你的金鑰
  （存成 Credential，別直接打在節點裡）

【Headers】Content-Type: application/json

【Body】開啟 Send Body，選 JSON：
{
  "inputs": {
    "transcript": {{ JSON.stringify($json.訪視紀錄) }},
    "has_foreign_caregiver":
        "{{ $json.外籍看護 }}",
    "income_status": "{{ $json.身分別 }}"
  },
  "query": "請根據訪視紀錄產出計畫。",
  "response_mode": "blocking",
  "user": "n8n-{{ $json.個案編號 }}"
}
```

預期回應（blocking 模式）

【Dify 回傳的內容】

```
{
  "event": "message",
  "task_id": "c3800678-...",
  "message_id": "9da23599-...",
  "conversation_id": "45701982-...",
  "mode": "advanced-chat",
  "answer": "【評估摘要】CMS 第 6 級…",
  "metadata": {
    "usage": { "total_tokens": 2140 },
    "retriever_resources": [ … ]
  },
  "created_at": 1705407629
}
```

★ 你要的評估報告在 answer 欄位：
{{ $json.answer }}

⋆ metadata.retriever_resources 是知識庫召回的原文段落——要做來源可追溯就靠它

三個欄位對應表： Dify 的變數 ↔ n8n 送出的值

Dify 表單變數 Key	類型	n8n 要送什麼	注意事項
transcript	段落 Paragraph	試算表中的訪視逐字稿欄位	用 JSON.stringify() 帶入，換行與引號才不會弄壞 JSON
has_foreign_caregiver	下拉 Select	必須是 Yes 或 No	★ 值不符合選項會被 Dify 拒絕；來源資料是「有／無」要先用 Set 節點轉換
income_status	下拉 Select	必須是 一般戶／中低收／低收	★ 同上，字串要完全一致，多一個空格都不行
query	（不是表單變數）	給 Chatflow 的指令句	對應 M8 手冊測試情境 A 的那句話
user	（不是表單變數）	任何能識別來源的字串	★ 必填；建議帶個案編號，方便日後在 Dify 後台追查

★ 最常見的整合失敗原因：下拉選項的值對不上。Dify 的 Select 型變數只接受你在表單裡設定的那幾個字串，不會自動轉換——所以 n8n 端要先用 Set 節點把來源資料正規化。

Step 3 ｜批次處理與寫回結果

開啟檔案

要組出的結構：

```
讀取試算表
      |
Loop Over Items
（Batch Size = 1）
      |
HTTP Request → Dify
      |
Set → 寫回試算表
```

預期結果

- ▸ Execute Workflow 後，能看到節點被重複執行 N 次（N ＝個案筆數）
- ▸ 試算表中每一列的評估報告欄位被逐一填上
- ▸ ★ 若中途某筆失敗，因為有 Retry 設定會自動重試三次，仍失敗才會中斷

操作步驟

1. 在讀取試算表節點後，加入「Loop Over Items （舊名 Split In Batches ）」節點
2. Batch Size 設 1—— 一次只送一筆給 Dify ，避免觸發並發上限
3. 把 Loop Over Items 的 loop 輸出接到 HTTP Request （呼叫 Dify ）
4. HTTP Request 的 Settings 分頁：開啟 Retry On Fail ， Max Tries 設 3 、 Wait Between Tries 設 3000 毫秒
5. 在 HTTP Request 後加一個 Set 節點，取出 {{ $json.answer }} 存成「評估報告」欄位
6. 再接 Google Sheets 節點（ Operation 選 Update Row ），把報告寫回原本那一列
7. ★ 最後把這一條路的末端接回 Loop Over Items 節點，形成迴圈，直到所有個案處理完

整合常見錯誤與排除

HTTP 狀態／訊息	意思	處方
401 unauthorized	金鑰錯誤或漏了 Bearer	檢查 Header 值格式： Bearer 空格 app-xxxx
400 not_chat_app	打錯 API 路徑	Chatflow 用 /chat-messages； Workflow 型應用才用 /workflows/run
400 app_unavailable	應用沒有發布	回 Dify 按「發布」，確認是已發布狀態
400 invalid_param	inputs 的變數 Key 或值不合法	對照上一頁的欄位對應表逐一檢查
429 too_many_requests	同時打太多請求	★ Batch Size 設 1，並加大 Wait Between Tries
回應被截斷或只回一小段	blocking 模式在部分版本／長回應下不穩	★ 改用 streaming 並在 n8n 端合併， 或把 Dify 的回應長度縮短
answer 是空字串	Chatflow 走到失敗分支	到 Dify 後台的「日誌與標註」看該次執行的節點狀態

★ 實務提醒： blocking 模式在長回應時可能被 Dify Cloud 的邊緣代理逾時中斷。若你的長照報告很長，建議先在 Dify 端把輸出改精簡，或改用 streaming 模式。

這個架構還能怎麼延伸

① 反過來：讓 Dify 的 Agent 呼叫 n8n

- 在 Dify 的 Agent 中新增自訂工具，指向 n8n 的 Webhook URL
- 適合：使用者在對話中要求「幫我把這份評估寄給王社工」——由 n8n 執行實際動作

② 加上人在迴路（呼應 M9）

- 評估金額超過某個門檻時，不直接寫回，而是先發通知請督導確認
- ★ 醫療與長照場域的黃金原則：機器做初步判斷，人做最終決定

③ 把來源可追溯性一起寫回

- 把 metadata.retriever_resources 裡的知識庫原文段落也寫進試算表
- ★ 這樣承辦可以自己核對「這個級別是根據哪一條規定判的」—— M9 的可解釋性要求

★ 這三個延伸方向，正好對應本課程的三個核心關切：自動化的效率（M8）、責任邊界（M9）、可解釋性（M9）。技術只是手段，把關機制才是專業。

進階預告： n8n 也能做 RAG

和 M8 學過的概念完全相通

- n8n 也有 Vector Store 、 Embeddings 、 Retriever 等節點，能自己組出檢索增強生成
- 切塊、嵌入、召回、生成—— M8 學過的四步驟，在 n8n 裡是四種不同節點的組合

什麼時候值得自己組，而不是直接用 Dify

- 當知識庫來源需要複雜前處理（先清洗、合併多個系統的資料）再送進向量資料庫
- 當你想把 RAG 檢索結果，接進更大、涉及多系統的自動化流程裡

課程定位

- 本講義聚焦 Agent 與工具串接的基本功； n8n 的 RAG 節點屬於進階延伸，原理已在 M8 學過

想深入研究的同學，可從 n8n 官方文件的 Vector Store 節點與 n8n Academy 的 RAG 課程開始，概念可直接沿用 M8 的知識。

常見問題 FAQ

Q： 一定要付費才能用 AI Agent 嗎?

▸ 不用。 Google Gemini 、 Groq 等都有免費額度； Ollama 本機模型完全免費

Q： n8n 和 Zapier ／ Make 選哪個?

▸ 簡單的兩端串接、不想碰任何設定，選 Zapier ／ Make ； 需要彈性、想自架、想接 AI Agent ，選 n8n

Q： Workflow 可以匯出給別人用嗎?

▸ 可以。 Workflow 選單 → Download 匯出 JSON （不含 Credential ），對方匯入後只需重設憑證

Q： Agent 會不會不小心花很多錢?

▸ 會——它可能重複呼叫工具與重新思考。務必設 Max Iterations 上限，並留意額度

延伸資源

官方資源

- n8n 官方文件： docs.n8n.io ｜ Dify API 文件： docs.dify.ai
- n8n Academy 互動課程： learn.n8n.io （有練習與證書）
- 社群模板庫： n8n.io/workflows—— 上千個現成 workflow 可直接匯入參考

與本課程的銜接

- M8 已教過的 Prompt 工程、 RAG 概念、 Agent 思考迴圈，在 n8n 中完全通用
- M9 的倫理與可解釋性原則（誠實面對不知道的事、數字不生成）同樣適用於 n8n Agent
- M10 教過的部署概念，若想公開你的 n8n workflow ，同樣需要考慮伺服器與網域

中英術語對照｜ n8n

（1 / 2）

中文	English	一句話說明
工作流程	Workflow	一群節點連接起來，從 Trigger 開始依序執行的整體
節點	Node	工作流程裡的一個方塊，代表一個動作或判斷
觸發節點	Trigger Node	決定 workflow 何時開始執行的特殊節點
憑證	Credential	存放 API 金鑰等認證資訊，供節點引用
執行紀錄	Execution	每跑一次 workflow 留下的一筆紀錄，除錯用
表達式	Expression	用兩層大括號動態帶入前面節點資料的語法
畫布	Canvas	編輯器中央拖曳節點、拉線的區域
節點詳細視窗	NDV, Node Detail View	雙擊節點後開啟的參數設定視窗
資料項	Item	節點之間傳遞的一筆資料；節點會對每一筆各執行一次

中英術語對照 | n8n

（2 / 2）

中文	English	一句話說明
子節點	Sub Node	接在 AI Agent 等根節點下方、擴充其能力的節點
根節點	Root Node	cluster node 中定義主要功能、可接子節點的節點
AI 代理人	AI Agent	能自行判斷下一步、呼叫工具完成任務的系統
鏈	Chain	固定順序呼叫 AI 元件，不具持久記憶
工具	Tool	Agent 可呼叫、用來與外部世界互動的資源
記憶	Memory	讓 Agent 記住先前對話內容的機制
迭代上限	Max Iterations	Agent 思考迴圈的最大輪數，用來控制成本
檢索增強生成	Retrieval-Augmented Generation, RAG	先檢索文件再生成，可標註來源（M8 已學）
範本	Template	官方或社群預先做好、可直接匯入的 workflow

本講義回顧：逐條自評

1. ☐ 在 Windows 上用 Docker 跑起 n8n，說出它的定位與六個核心概念
2. ☐ 看懂編輯器、節點與資料流，會用 Expression
3. ☐ 完成範例一： Trigger → 抓資料 → 判斷 → 輸出
4. ☐ 做出多工具 AI Agent，用 $fromAI() 把任意 API 變成工具
5. ☐ 用 Executions 除錯、正確發布；能把 Dify 長照 3.0 接進 n8n

★ 背景： n8n 版本更新很快（3.0 正式版預定 12 月）； Trigger → 處理 → 輸出的觀念不變

下一步　挑一個你自己真的會用到的流程，實際做一個屬於自己的 workflow—— 這是唯一能真正學會的方法。

總複習｜安裝與核心概念

Q 1

Windows 安裝 n8n 需要先裝哪兩個東西?

WSL2 與 Docker Desktop；不需要安裝 Node.js。

Q 2

一個 workflow 至少要有哪一種節點才能執行?

至少一個 Trigger 節點，決定何時開始執行。

Q 3

上一個節點輸出 10 筆 item，下一個節點會執行幾次?

10 次——節點會對每一筆 item 各執行一次。

Q 4

Workflow 一直失敗，第一步該去哪裡看?

Executions 執行紀錄，點開失敗那筆逐節點檢查輸入與輸出。

總複習 | AI Agent 與整合

Q1

AI Agent 節點必填的兩個子節點插槽是什麼?

Chat Model 與至少一個 Tool； Memory 為選填但建議接上。

Q2

$fromAI() 的作用與限制是什麼?

讓 Agent 自己決定傳給工具的參數值；只在 Agent 的 Tool 節點中有效。

Q3

n8n 呼叫 Dify 的 Chatflow，該打哪一個 API 路徑?

POST /chat-messages，並在 inputs 帶入開始節點的表單變數。

Q4

批次呼叫 Dify 為何要 Batch Size 設 1？

避免同時送出大量請求觸發並發上限（429 too_many_requests）。

Q5

需求是「串接十個系統做自動化」，選哪個?

n8n—— 強項是通用服務串接與流程自動化。

課後複習：本單元的互動學習站

掃我開啟本教材互動版

手機相機對準即可開啟

網址

nchsieh.github.io/assets/courses/001/interactive/n8n/

13 節，順序對應講義各 PART

看完一個 PART 就做對應的那一節，不必一次做完

改參數、立刻看結果

圖表可以拖拉調整，看懂「為什麼」，而不只是背結論

隨堂練習＋即時回饋

答錯會說明原因；作答進度只存在你自己的瀏覽器

全部單元入口： nchsieh.github.io/assets/courses/001/interactive/ · 網站內容為教學示例與模擬資料，不需登入